

REPUBLIC OF CAMEROON

PEACE – WORK –FATHERLAND

MINISTRY OF HIGH SCHOOL
EDUCATION
UNIVERSITY INSTITUTE OF
THE COAST

REPUBLIQUE DU CAMEROUN

PAIX – TRAVAIL-PATRIE

MINISTRE DEL'ENSEIGNEMENT
SUPERIEUR
INSTITUT UNIVERSITAIRE DE LA
COTE



DESIGN PROJECT

MONITORING SUSPICIOUS DISCUSSIONS ON ONLINE PLATFORMS USING MACHINE LEARNING

*An academic project report submitted to the Department of Engineering, SEAS-IUC, in
partial fulfilment of the requirements for the award of a Bachelor of Technology (B. Tech)
degree in*

COMPUTER SCIENCE ENGINEERING (CSE)

Specialty: FULL STACK DEVELOPMENT

Written and presented by:

FONGANG NGNIJIKO HELENE GRACE

FOSSO TEMFACK MIRENDA CASSIDY

Supervised by:

Mr. SOBGOU FABRICE

2025-2026 ACADEMIC YEAR

CERTIFICATION

This is to certify that this report titled: **MONITORING OF SUSPICIOUS DISCUSSIONS ON ONLINE PLATFORMS USING MACHINE LEARNING** written and presented by **FONGANG NGNIJIKO HELENE GRACE** and **FOSSO TEMFACK MIRENDA CASSIDY** in partial fulfilment of the requirements and regulations governing the award of the **BACHELOR OF TECHNOLOGY in FULLSTACK DEVELOPMENT** at University Institute of the Coast.

Supervisor's name	Signature	Date
Mr. SOBGOU FABRICE		

Head of Department	Signature	Date
Mr. FONKOU LESTER CONFIANCE		

DEDICATION



ACKNOWLEDGEMENTS

This work could not have been completed without the moral and intellectual support of several important individuals. The realization of this research project is the result of laborious and dedicated academic effort. We take this opportunity to express our sincere thanks to:

- ❖ **Mr. GUIMEZAP PAUL**, Founder and President of IUC, for all the means employed with the goal of providing a favourable environment and condition for our studies;
- ❖ **Dr. TCHIKJE PRUDENCE**, the Director of SEAS, for his sense of management, listening, and accompaniment;
- ❖ Our heartfelt gratitude goes to my academic supervisor, **MR. SOBGOU FABRICE**, for his moral and intellectual guidance, valuable advice, and encouragement throughout this work;
- ❖ We also sincerely thank all my lecturers of the Anglophone section of IUC, especially our Head of Department, **Mr. Fonkou Lester Confiance**, who, despite his busy schedule, made time to support the completion of this report.
- ❖ We are equally grateful to my friends and family for their continuous motivation and encouragement. Finally, we extend my deepest thanks to the Creator, the giver of life, for the strength and perseverance granted to me during this research period.

TABLE OF CONTENT

CERTIFICATION	ii
DEDICATION	iii
ACKNOWLEDGEMENTS	iv
TABLE OF CONTENT	v
LIST OF TABLES	vii
LIST OF FIGURES	viii
LIST OF ABBREVIATIONS	ix
GENERAL INTRODUCTION	1
I. Background Study	1
II. Understanding Suspicious Discussions (Hate Speech)	1
1. Definition	1
2. Impact of Suspicious Online Discussions	2
III. Problem Statement	2
IV. Objective of the System	3
1 General Objective	3
2 Specific Objectives	3
V. Justification	3
VI. Work Plan	4
CHAPTER ONE: EXISTING ALGORITHMS USED IN MONITORING OF SUSPICIOUS DISCUSSIONS ON ONLINE PLATFORMS	6
1.1 INTRODUCTION	6
1.2 General Architecture of ML-Based Monitoring Systems	6
1.3 Machine Learning Algorithms for Hate Speech Detection	8
1.3.1 Naive Bayes (NB)	8
1.3.2 Support Vector Machine (SVM)	9
1.3.3 Random Forest (RF)	11
1.1.1 Logistic Regression (LR)	13
1.4 Comparative Study of Algorithms	14
1.5 Limitation of Existing Hate Speech Detection Systems	15
1.6 Proposed Solution	16
Conclusion	16
CHAPTER TWO: SYSTEM ANALYSIS AND ML METHODOLOGY	17
2.1 Introduction	17
2.2 Methodology	17
2.3 MLOps Lifecycle	17

2.4	Machine Learning Pipeline Design	19
2.4.1	Data collection (using Web Scraping)	19
2.4.2	Data Preprocessing	24
2.4.3	Feature Extraction with TF-IDF	29
2.4.4	Model Selection (Classification)	33
2.4.5	Model Training	36
2.4.6	Model Evaluation	40
2.5	SYSTEM ARCHITECTURE	43
2.6	API Design	44
2.6.1	API Architecture	44
2.6.2	Core API Endpoint	45
2.7	Database Design	45
2.7.1	The Nature of Document-Oriented NoSQL	45
2.7.2	Why NoSQL is Essential for Scraped ML Data.....	46
Conclusion.....		47
REFERENCES		48

LIST OF TABLES

Table 1: Work Plan	4
Table 2 : Advantages of Naive Bayes	9
Table 3: Limitations of Naïve Bayes	9
Table 4: Advantages of SVM	11
Table 5: Limitations of SVM.....	11
Table 6: Advantages of Random Forest.....	12
Table 7:Limitations of Random Forest.....	13
Table 8:Advantages of Logistic Regression	14
Table 9: Limitations of Logistic Regression	14
Table 10: Comparative Study of Algorithms.....	15
Table 11: ML Lifecycle Phases Mapped to MLOps Stages [Sculley et al., 2015].....	18
Table 12: Data schema table.....	22
Table 13: TF-IDF Vectorization Parameters.....	32
Table 14:: Data Split Distribution.....	37
Table 15: SVM Training Configuration	39
Table 16: Confusion Matrix Structure (2-Class)	40
Table 17: Evaluation Metrics.....	42
Table 18: System component description.....	43

LIST OF FIGURES

Figure 1: Categories of Hate Speech	2
Figure 2 : Gantt Diagram.....	5
Figure 3: Machine learning system flowchart	7
Figure 4: Naive Bayes flowchart.....	8
Figure 5: Support Vector Machine flowchart	10
Figure 6: Random Forest flowchart.....	12
Figure 7: Logistic Regression flowchart	13
Figure 8: ML Life Cycle Diagram.....	18
Figure 9: Web Scrapping cycle.....	20
Figure 10: Web Scrapping workflow diagram	21
Figure 11: Data collection workflow diagram.....	23
Figure 12: Data Preprocessing workflow diagram	27
Figure 13: Missing Value Handling Process in Scraped Dataset	29
Figure 14: Feature Extraction with TF-IDF workflow	31
Figure 15:: Support vector machine workflow.....	36
Figure 16: Cross validation	38

LIST OF ABBREVIATIONS

ML: Machine Learning

AI: Artificial Intelligence

NLP: Natural Language Processing

API: Application Programming Interface

SVM: Support Vector Machine

HTTP: HyperText Transfer Protocol

REST: Representational State Transfer

URL: Uniform Resource Locator

SQL: Structured Query Language

F1: F1-Score (Harmonic Mean of Precision and Recall)

TF-IDF: Term Frequency-Inverse Document Frequency

NB: Naïve Bayes

GENERAL INTRODUCTION

I. Background Study

Social media platforms such as Facebook, Twitter, and Instagram have significantly transformed communication and cultural interactions worldwide [1]. These platforms allow users to share opinions, emotions, and ideas on topics such as gender, race, ethnicity, religion, and politics. However, their accessibility, anonymity, and rapid information dissemination have also facilitated the spread of harmful content, particularly hate speech [2].

Hate speech is generally defined as any form of expression that attacks, threatens, or discriminates against individuals or groups based on protected attributes such as race, religion, gender, or ethnicity [3]. Such content often promotes hostility and can incite violence against targeted communities. Due to the viral nature of social media, hate speech can spread rapidly, amplifying conflicts both locally and globally [4].

The presence of hate speech online has serious consequences, including psychological harm to individuals and increased social tension. Toxic online discourse can disrupt communities and contribute to societal instability. Studies have shown that hate speech is complex and multi-layered, often involving implicit meanings and contextual nuances that make detection difficult [5].

Traditional approaches to identifying harmful content relied on manual moderation. However, the large volume of user-generated data makes this approach inefficient and not scalable. As a result, automated techniques have become necessary [6].

Machine Learning (ML) and Natural Language Processing (NLP) techniques are increasingly used to detect hate speech in textual data. These approaches enable systems to learn patterns from labeled datasets and classify text into categories such as hate speech and normal content. Various studies have explored the use of machine learning models, including Support Vector Machines (SVM), for hate speech detection tasks [7].

II. Understanding Suspicious Discussions (Hate Speech)

1. Definition

Hate speech refers to any form of expression that attacks, demeans, or incites violence against individuals or groups based on protected characteristics such as race, religion, ethnicity,

gender, sexual orientation, or disability [2]. According to [5], hate speech is multilayered, complex, and incendiary information that targets particular individuals or groups.

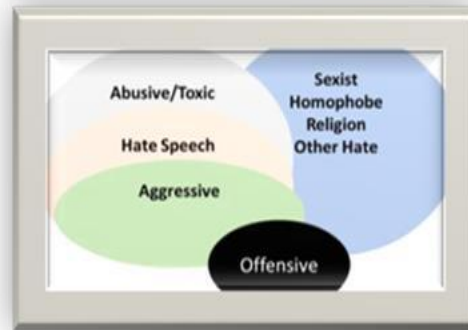


Figure 1: Categories of Hate Speech

2. Impact of Suspicious Online Discussions

The presence of hate speech on online platforms has documented negative consequences [2][6]:

- ❖ **Psychological harm:** Victims experience anxiety, depression, and emotional distress
- ❖ **Social division:** Hate speech reinforces stereotypes and polarizes communities
- ❖ **Real-world violence:** Online hate speech has been linked to offline hate crimes
- ❖ **Chilling effect:** Targeted users withdraw from online participation
- ❖ **Platform degradation:** Unmoderated hate speech reduces user trust and engagement

III. Problem Statement

- ❖ Despite the rapid growth of online communication platforms and the increasing prevalence of hate speech, automated detection remains a significant technical challenge. Victims of online hate speech often experience psychological distress, while the spread of toxic content discourages user participation and damages platform reputation [2]. Derived from this main problem are the following specific technical obstacles:
- ❖ **Poor Data Quality and Unstructured User-Generated Content:** Online text data is often noisy, inconsistent, and unstructured, making it difficult to process and analyze

effectively. User-generated content contains special characters, URLs, mentions, misspellings, and inconsistent capitalization, all of which degrade machine learning model performance if not properly addressed [8].

- ❖ **Difficulty in Representing Text Data for Machine Learning:** Raw textual data cannot be directly used by machine learning models. Converting text into meaningful numerical representations that accurately capture word importance and semantic meaning remains a fundamental challenge in hate speech detection [9].
- ❖ **Inaccurate Detection Due to Context and Nuance:** Hate speech detection requires understanding sarcasm, irony, coded language, and cultural context. Simple keyword-based filters fail to distinguish between genuinely hateful content and benign language. This leads to high false positive rates and false negative rates [5].
- ❖ **Class Imbalance in Hate Speech Datasets:** Hate speech is relatively rare compared to normal content on most online platforms. This class imbalance causes machine learning models to become biased toward predicting "normal" for all inputs, achieving high accuracy while failing completely at detecting hate speech [10].

IV. Objective of the System

1 General Objective

The general objective of this project is to develop and evaluate a machine learning-based model for detecting hate speech on online platforms using supervised classification techniques.

2 Specific Objectives

- ❖ **Data Collection:** Collect publicly available labeled hate speech datasets from reputable sources such as Jigsaw, Davidson et al. (2017), and ETHOS.
- ❖ **Preprocessing:** Preprocess raw text data by applying cleaning operations including lowercasing, removing special characters, removing URLs and mentions, removing stopwords, tokenization, and stemming.
- ❖ **Feature Extraction:** Convert text data into numerical features using TF-IDF (Term Frequency-Inverse Document Frequency) to represent word importance for classification.

V. Justification

Monitoring and detecting hate speech on online platforms is important due to several factors. First, hate speech affects millions of users globally, causing real psychological and social harm

including anxiety, depression, and emotional distress that cannot be ignored [2][6]. Second, social media platforms are increasingly held legally and ethically accountable for harmful content on their services, requiring effective content moderation systems [3]. Third, manual moderation is no longer sufficient because human reviewers cannot review millions of messages posted daily at scale; manual approaches are slow, expensive, inconsistent, and psychologically damaging [8]. Fourth, simple keyword filtering and rule-based systems fail to understand context, sarcasm, and evolving hate speech vocabulary, producing high false positive rates and high false negative rates [9]. Fifth, existing comparative studies have identified SVM as the best-performing classifier for hate speech detection, achieving the highest F1-scores across multiple datasets [7][11]. Sixth, hate speech is rare compared to normal content, causing machine learning models to become biased toward predicting "normal" for all inputs, which this project addresses explicitly through class imbalance techniques [10]. Finally, this project follows standard ML practices including cross-validation, hyperparameter tuning, and clear evaluation protocols, ensuring that results are reproducible and transparent [12].

VI. Work Plan

Table 1: Work Plan

Task Description	Start Date	End Date	Duration
Determine project objectives, goals, and scope	30/01/2026	07/02/2026	1 week
Requirement gathering	08/02/2026	15/02/2026	1 week
Conduct research on existing hate speech/harassment detection systems	16/02/2026	27/02/2026	1 week
Choosing requirements and assemble all components	28/02/2026	06/03/2026	1 week
Design system architecture	16/03/2026	26/03/2026	1 week
Build custom platform (frontend + backend)	30/03/2026	17/04/2026	3 weeks
Train ML model	18/04/2026	25/04/2026	2 weeks

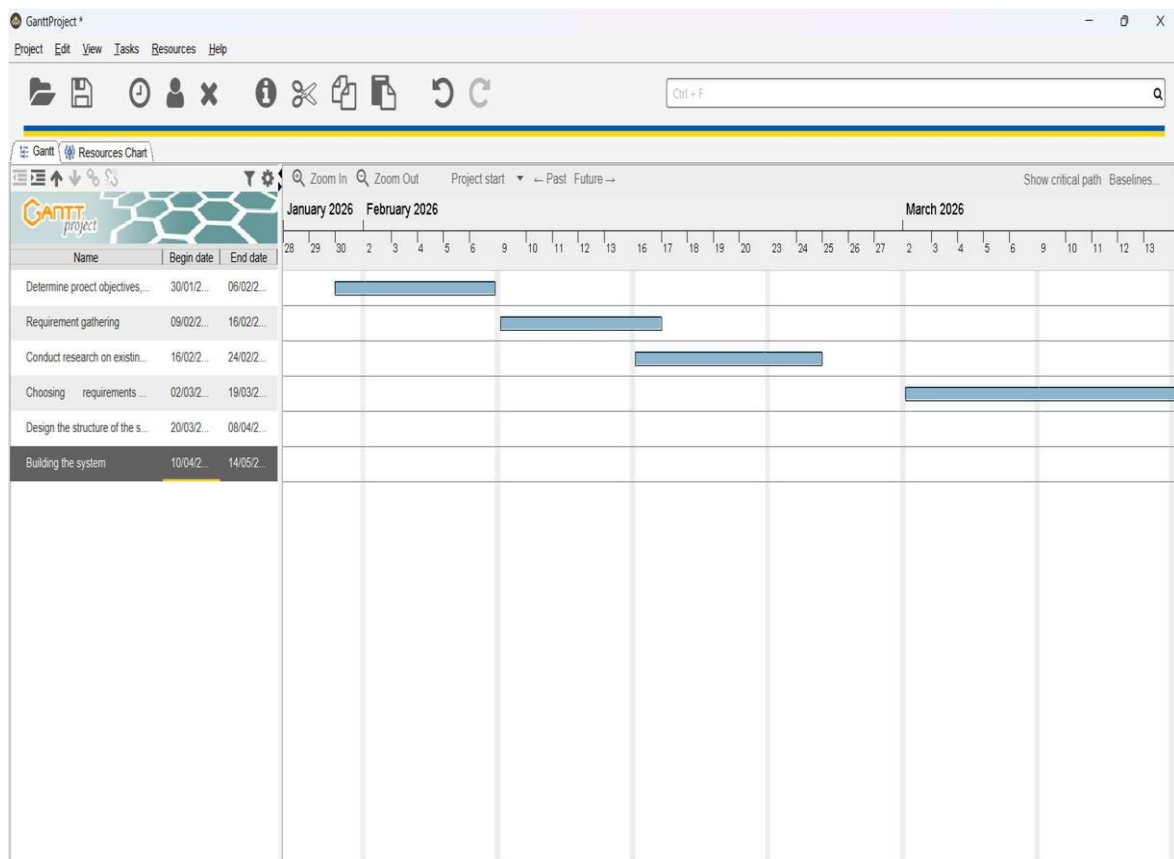


Figure 2 : Gantt Diagram

CHAPTER ONE: EXISTING ALGORITHMS USED IN MONITORING OF SUSPICIOUS DISCUSSIONS ON ONLINE PLATFORMS

1.1 INTRODUCTION

The rapid growth of online communication platforms has significantly transformed how individuals interact, share opinions, and participate in social discourse. However, this transformation has also led to the widespread emergence of hate speech, which negatively impacts individuals and communities, often leading to psychological distress, social division, and in extreme cases, real-world violence [2][5].

Traditional moderation techniques, including manual review and rule-based filtering systems, are no longer sufficient due to the massive volume of user-generated content produced daily. Consequently, Machine Learning (ML) has emerged as a powerful approach for automating the detection of hate speech by learning patterns from data and making predictive decisions [8].

In ML-based text classification systems, raw textual data is transformed into structured numerical representations through feature extraction techniques such as TF-IDF, after which classification algorithms are applied to categorize messages into classes such as normal or hate speech [9][10]. This chapter explores the existing machine learning algorithms used in hate speech detection systems, highlighting their operational mechanisms, advantages, and limitations.

1.2 General Architecture of ML-Based Monitoring Systems

Most machine learning systems for detecting suspicious discussions follow a structured pipeline

as described in prior studies [6][7]:

- ❖ **Data Collection:** Collection of textual data from online platforms such as forums, chats, or social media.
- ❖ **Data Preprocessing:** Cleaning operations including tokenization, stop-word removal, stemming, and normalization.

- ❖ **Feature Extraction:** Conversion of text into numerical vectors using techniques such as TF-IDF.
- ❖ **Model Training:** Application of ML classifiers such as SVM, Naïve Bayes, or Random Forest
- ❖ **Classification:** Assigning labels such as *hate speech* or *normal*.
- ❖ **Evaluation:** Performance measured using Accuracy, Precision, Recall, and F1-score.

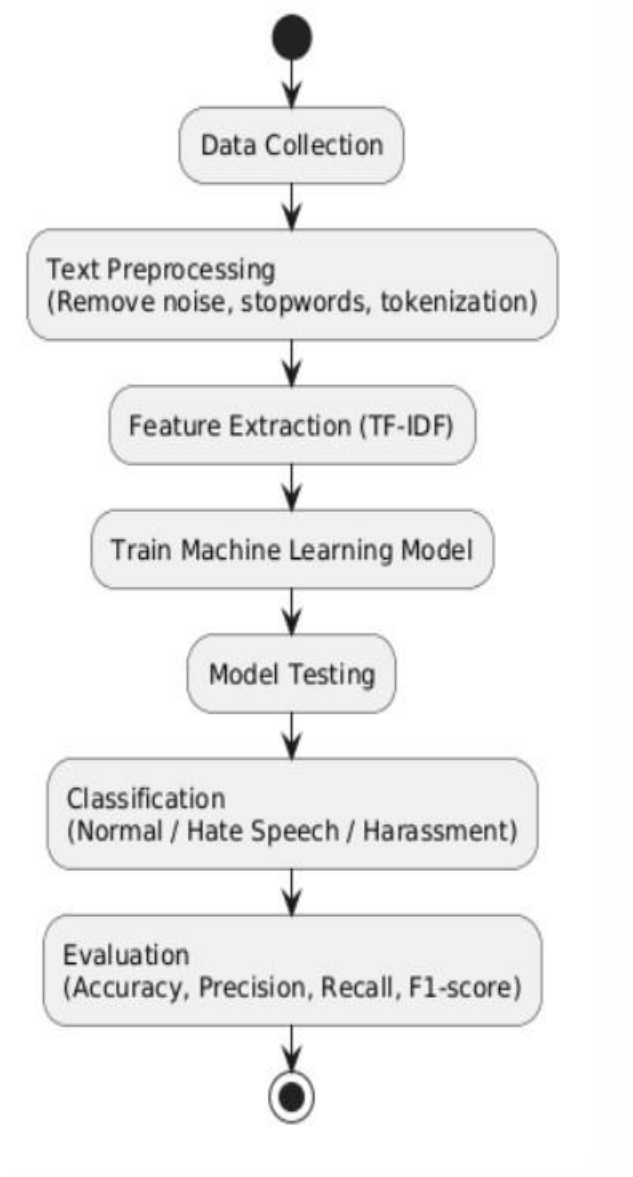


Figure 3: Machine learning system flowchart

1.3 Machine Learning Algorithms for Hate Speech Detection

1.3.1 Naive Bayes (NB)

Naive Bayes is a probabilistic classifier based on Bayes' Theorem, assuming independence among features. It is widely used in text classification tasks due to its simplicity and efficiency [13].

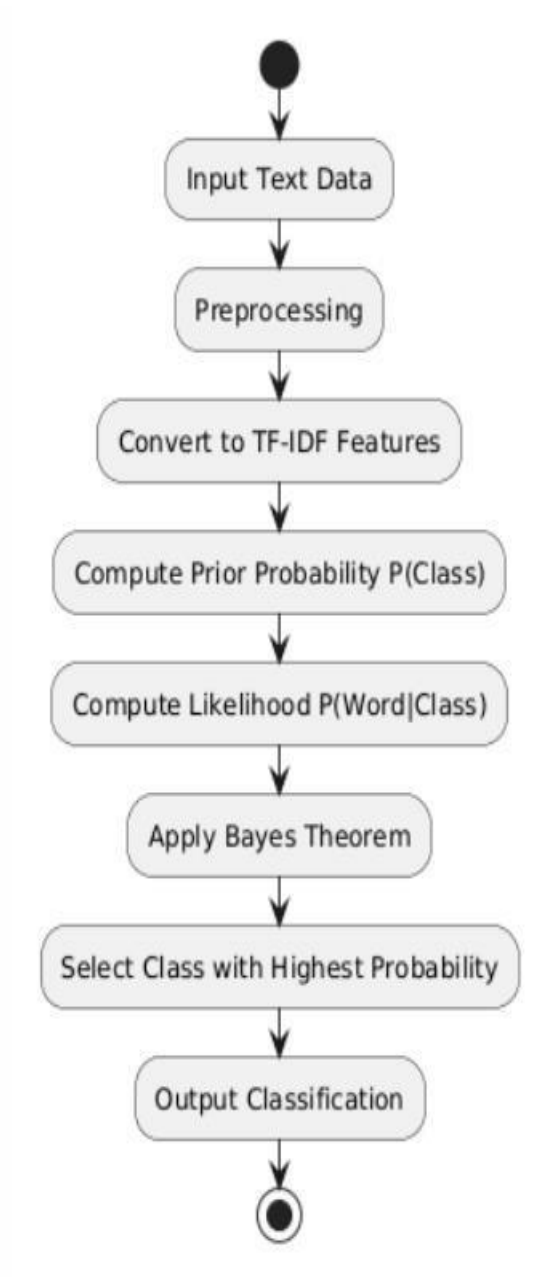


Figure 4: Naive Bayes flowchart

❖ **Advantages***Table 2 : Advantages of Naive Bayes*

Advantage	Description
Simplicity	Easy to implement and computationally efficient
Fast Training	Requires less training time compared to complex models
Low computational cost	Suitable for real-time applications
Works well with text data	Performs well with high-dimensional datasets like TF-IDF

❖ **Limitations***Table 3: Limitations of Naïve Bayes*

Limitation	Description
Independence assumption	Assumes features are independent, which is unrealistic in language
Lower accuracy	Performs poorly when feature relationships are important
Poor context understanding	Cannot capture semantic relationships between words
Sensitive to data quality	Noise in data affects performance significantly

1.3.2 Support Vector Machine (SVM)

Support Vector Machine is a supervised learning algorithm that finds an optimal hyperplane to separate data into different classes [12]. It is highly effective in text classification tasks because it handles high-dimensional feature spaces efficiently.

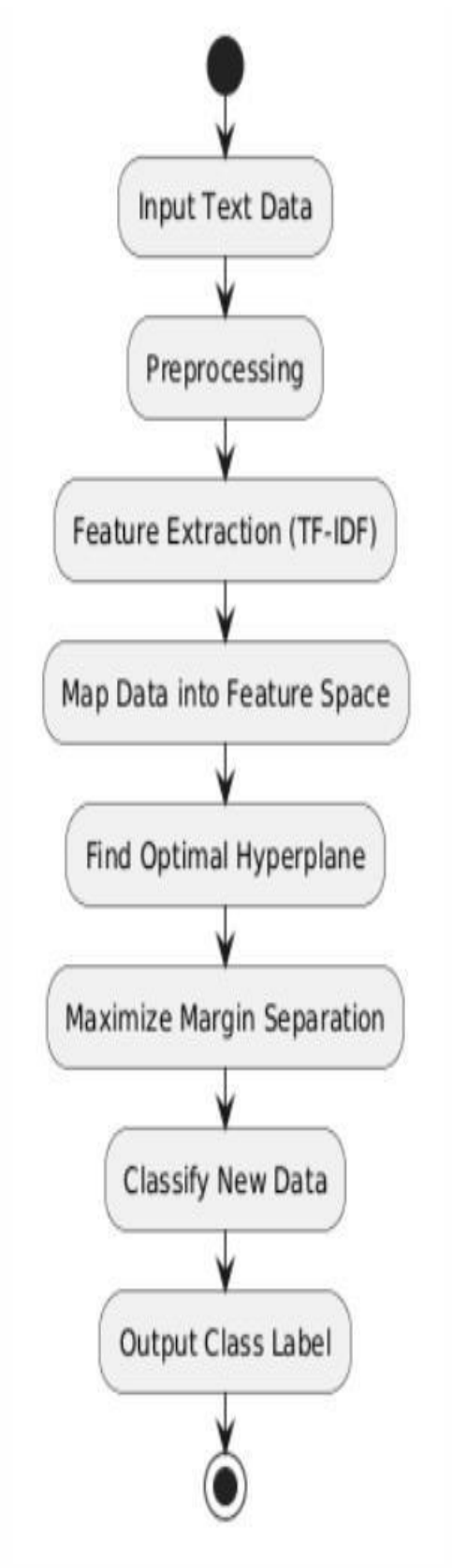


Figure 5: Support Vector Machine flowchart

❖ **Advantages***Table 4: Advantages of SVM*

Advantage	Description
High accuracy	Performs well in text classification tasks
Effective in high dimensions	Handles large feature sets like TF-IDF vectors
Robust to overfitting	Works well even with limited data

❖ **Limitations***Table 5: Limitations of SVM*

Limitation	Description
Memory intensive	Needs significant memory for large datasets
High computational cost	Training can be slow for large datasets
Difficult to interpret	Hard to explain decision boundaries
Requires parameter tuning	Kernel selection impacts performance

1.3.3 Random Forest (RF)

Random Forest is an ensemble learning algorithm that combines multiple decision trees and makes predictions based on majority voting [14]. It reduces overfitting and improves classification performance by aggregating multiple models.

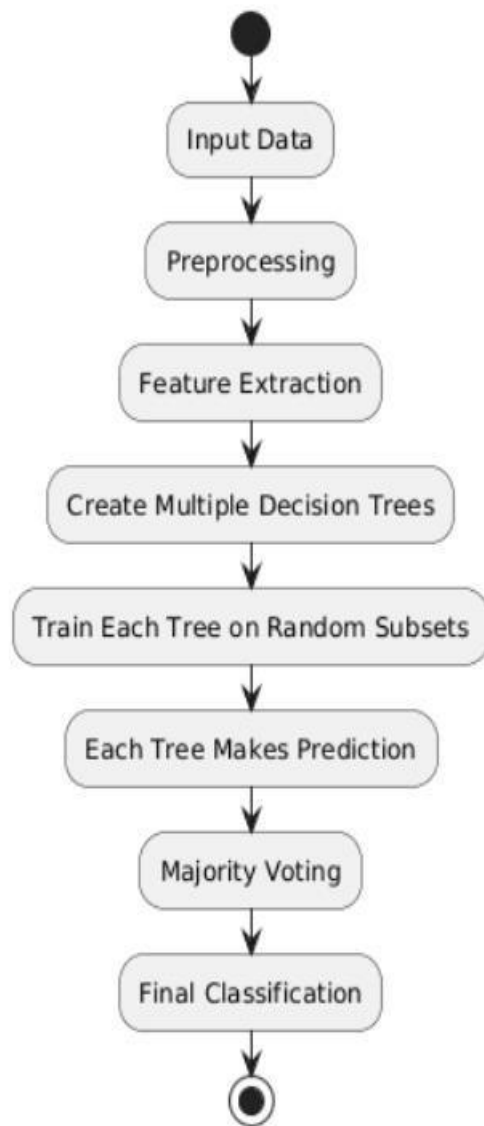


Figure 6: Random Forest flowchart

❖ Advantages

Table 6: Advantages of Random Forest

Advantage	Description
High accuracy	Performs well across different datasets
Feature importance	Identifies important features
Handles overfitting	Reduces variance using multiple trees
Works with large data	Can handle large datasets efficiently

❖ Limitations

Table 7: Limitations of Random Forest

Limitation	Description
Computational complexity	Requires more resources than simple models
Less interpretable	Difficult to understand compared to single tree
Slower prediction	Compared to simpler models
Requires tuning	Needs optimization of parameters

1.1.1 Logistic Regression (LR)

Logistic Regression is a statistical model used for binary classification problems. It uses a sigmoid function to model probability [15].

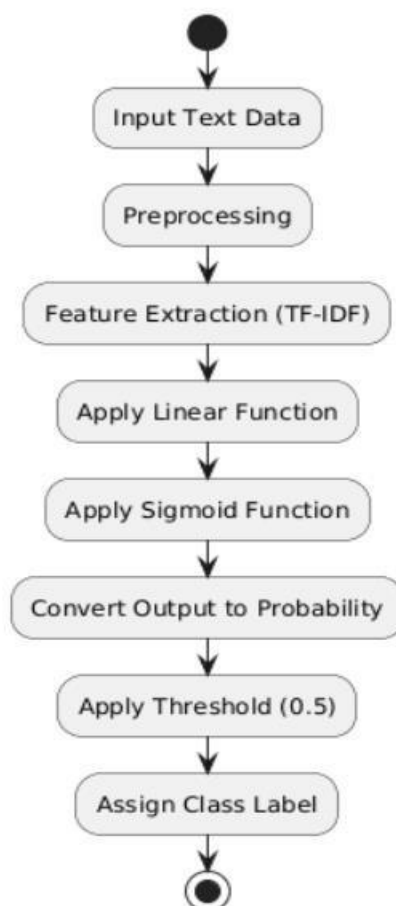


Figure 7: Logistic Regression flowchart

❖ **Advantages***Table 8: Advantages of Logistic Regression*

Advantage	Description
Easy to implement	Simple and efficient
Interpretable	Coefficients show feature importance
Fast computation	Suitable for large datasets
Works well for linear problems	Works well for linear problems

❖ **Limitations***Table 9: Limitations of Logistic Regression*

Limitation	Description
Linear assumption	Cannot handle complex relationships
Sensitive to outliers	Performance affected by noisy data
Lower performance	Compared to advanced models
Requires feature engineering	Needs well-prepared input

1.4 Comparative Study of Algorithms

Several studies have evaluated machine learning algorithms for detecting hate speech [7][11][16]:

Study	Algorithm	Performance
Burnap & Williams (2015)	SVM	67% F1-score
Greevy et al. (2008)	SVM	87% accuracy
Kwok & Wang (2013)	Naïve Bayes	76% accuracy
Waseem & Hovy (2016)	Logistic Regression	73% F1-score
Nobata et al. (2016)	SVM	77% F1-score
Dinakar et al. (2011)	Multiple models	73% accuracy

Recent comparative research shows the following performance patterns [7]:

Table 10: Comparative Study of Algorithms

Advantage	Description		
Algorithm	Accuracy	F1-Score	Observation
SVM	~79%	~0.77	Best overall performance
Random Forest	~75%	~0.74	Stable performance
Logistic Regression	~75%	~0.72	Good baseline
Naïve Bayes	~73%	~0.68	Fast but less accurate
KNN	~57%	~0.47	Poor performance
AdaBoost	~78%	~0.73	Competitive

Evaluation is primarily based on:

- ❖ **Accuracy:** Overall correctness
- ❖ **Precision:** Correct harmful detections
- ❖ **Recall:** Ability to detect all harmful content
- ❖ **F1-score:** Balance between precision and recall

1.5 Limitation of Existing Hate Speech Detection Systems

Despite the progress made in hate speech detection using machine learning, existing systems suffer from several limitations [2][7][8].

First, most studies evaluate only one or two machine learning algorithms on their own chosen datasets, making it impossible to fairly compare which model actually performs best [7]. Second, there is no standardization in preprocessing techniques. Some studies apply aggressive cleaning that removes important contextual information, while others apply minimal cleaning that leaves noise in the data [8]. Third, many existing systems ignore the class imbalance problem. Hate speech is rare compared to normal content, yet many models are trained on imbalanced datasets without any mitigation technique [10]. Fourth, evaluation metrics are inconsistently reported. Some studies report only accuracy, which is misleading for imbalanced datasets, while others report F1-score but use different averaging methods [6]. Fifth, hyperparameter tuning is often poorly documented or not performed at all, raising concerns about reproducibility [12].

1.6 Proposed Solution

Based on the limitations identified above, this project will implement a hate speech detection model using the Support Vector Machine (SVM) classifier, which has been identified as the best-performing algorithm in comparative literature studies [7][11]. The implementation will follow standardized preprocessing, consistent evaluation metrics, and proper class imbalance handling. All preprocessing steps and hyperparameters will be documented to ensure reproducibility. Class imbalance will be addressed using class weighting techniques. This approach ensures a reliable and transparent hate speech detection model.

1.7 Conclusion

This chapter reviewed existing machine learning algorithms used for hate speech detection. Four algorithms were examined: Naïve Bayes, Support Vector Machine (SVM), Random Forest, and Logistic Regression. Comparative studies showed that SVM consistently achieves the highest F1-score (~ 0.77) for hate speech detection tasks, followed by Random Forest (~ 0.74) and Logistic Regression (~ 0.72) [7][11]. Based on these findings, SVM was selected as the most suitable algorithm for this project due to its ability to handle high-dimensional TF-IDF features and its robust performance across multiple studies.

CHAPTER TWO: SYSTEM ANALYSIS AND ML METHODOLOGY

2.1 Introduction

This chapter presents the complete system analysis and design for developing a hate speech detection model using ML. The chapter strictly follows the MLOps lifecycle structure which are covering data collection, data preprocessing, feature extraction, model selection, model training, and model evaluation. The methodology is designed to be reproducible and follows standard machine learning practices [12][17].

2.2 Methodology

To address the objectives of this project, a structured methodology based on the Machine Learning Operations (MLOps) lifecycle was adopted. This approach ensures a systematic development process for monitoring suspicious discussions on online platforms, covering all stages from data collection and preprocessing to feature engineering, model selection, training, evaluation, and deployment. By following this framework, the system is designed to achieve accuracy, scalability, and consistency in analyzing and classifying textual data [16].

2.3 MLOps Lifecycle

The machine learning subsystem follows MLOps principles, which combine ML development with operational best practices to ensure models are built, evaluated, deployed, and continuously improved in a reliable and reproducible manner [Sculley et al., 2015]. The lifecycle used in this project consists of four stages and eight phases:

Table 11: ML Lifecycle Phases Mapped to MLOps Stages [Sculley et al., 2015]

Phase	MLOps Stage	Description
Problem Definition	Scoping	Define the binary classification task (hate speech vs. normal)
Data Collection	Data Engineering	Gather labeled hate speech datasets from public sources
Data Preprocessing	Data Engineering	Clean, normalize, and transform raw text into analysis-ready format
Feature Extraction	Data Engineering	Convert text into numerical vectors using TF-IDF
Model Training	Modelling	Train the selected model and evaluate using F1, Precision, Recall, Accuracy
Model Evaluation	Operations	Integrate into the backend and monitor performance in production

*Figure 8: ML Life Cycle Diagram*

2.4 Machine Learning Pipeline Design

The Machine Learning (ML) pipeline constitutes the analytical backbone of the Hate Speech Detection System. Following the formal methodology described in [16], the pipeline systematically transforms unstructured textual data collected from online platforms into structured numerical representations that can be processed by machine learning algorithms. This transformation enables the system to objectively analyze linguistic patterns and classify messages into predefined categories.

The pipeline is designed as a sequential process, beginning with automated data acquisition through web scraping, followed by data preprocessing, feature extraction, model training, evaluation, and deployment. Each stage ensures that raw textual data is progressively refined into a format suitable for accurate and reliable classification.

2.4.1 Data collection (using Web Scraping)

2.4.1.1 Data Sources

Since the system monitors real-world discussions, data is collected from publicly accessible online platforms where user interactions occur.

Primary Data Sources

- ❖ Public social media pages (discussion threads, comments)
- ❖ Online forums and discussion boards
- ❖ News website comment sections
- ❖ Public group discussions

These platforms are selected because they contain:

- ❖ High volumes of user-generated text
- ❖ Real conversational language
- ❖ Presence of harmful and non-harmful interactions

2.4.1.2 Web Scraping Mechanism

Web scraping is used to automatically extract textual data from web pages.

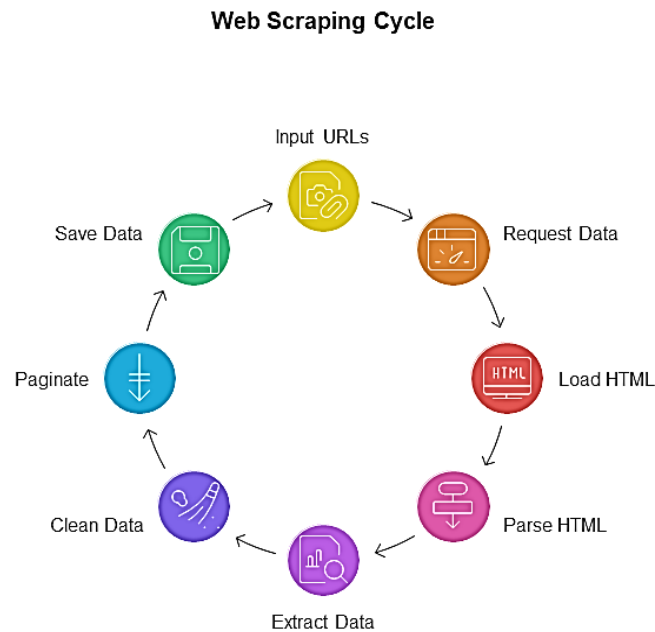


Figure 9: Web Scraping cycle

2.4.1.3 Operation process

- ❖ Access a target web page containing discussions
- ❖ Retrieve the page structure (HTML content)
- ❖ Identify sections containing user messages
- ❖ Extract textual content
- ❖ Store extracted data for further processing

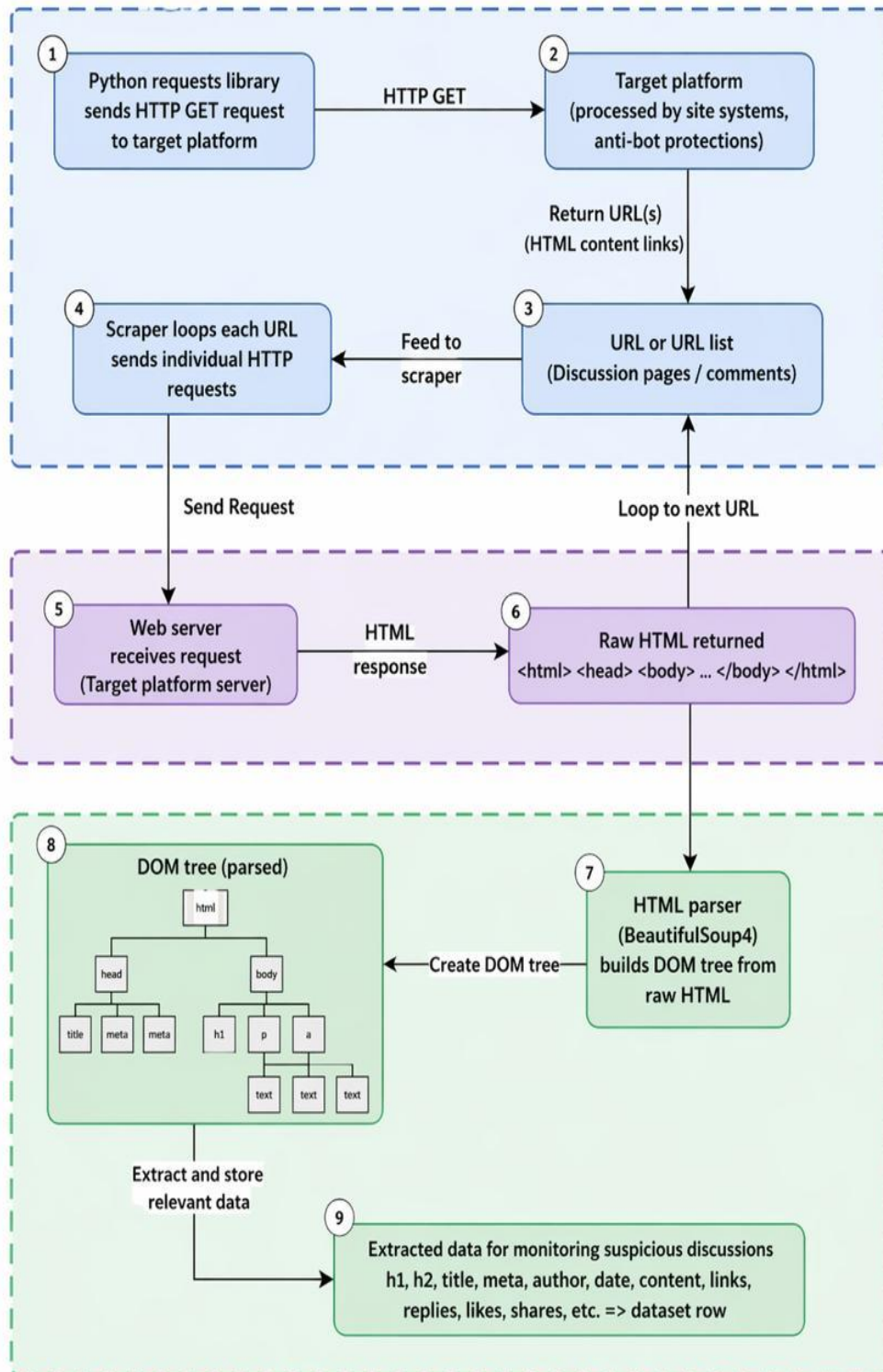


Figure 10: Web Scrapping workflow diagram

2.4.1.4 Data Structure Design

To ensure consistency, scalability, and efficient data management, the collected data is organized using a well-defined database schema. The structure is designed to support data storage, preprocessing, labeling, and machine learning tasks.

Each data instance represents a single text entry (e.g., a comment, post, or message) and is stored as a record in the database. The schema includes the following core fields:

Table 12: Data schema table

Column Name	Description
message_id	Unique identifier for each message
text	Raw message content
source	Platform or website where data was collected
timestamp	Date and time of message
language	Detected language of the message
label	Assigned class (Normal / Harassment / Hate Speech)

2.4.1.5 Explanation of Structure

- ❖ **message_id** : ensures traceability
- ❖ **text** : contains the raw input for the ML model
- ❖ **source** : allows analysis across platforms
- ❖ **timestamp** : supports temporal analysis
- ❖ **language** : helps manage multilingual data
- ❖ **label** : is required for supervised learning

2.4.1.6 Data Annotation Process

Since the data collected through web scraping is inherently unlabeled, a manual annotation process is required to assign meaningful class labels for hate speech detection. In this study, each message is independently reviewed by two annotators to ensure objectivity and minimize individual bias during labeling. The annotators assign labels to each text instance based on predefined annotation guidelines that clearly define categories such as hate speech, offensive language, and neutral content. In cases where disagreements arise between annotators, these are

resolved through discussion and consensus to ensure the final label accurately reflects the content. To assess the reliability and consistency of the annotation process, inter-annotator agreement is measured using Cohen's Kappa coefficient [21], which evaluates the level of agreement beyond chance. The coefficient is calculated as follows:

$$\kappa = \frac{P_o - P_e}{1 - P_e}$$

where P_o represents the observed agreement between annotators, and P_e represents the expected agreement by chance. This measure provides a robust indication of annotation quality and ensures that the labeled dataset is reliable for subsequent machine learning tasks.

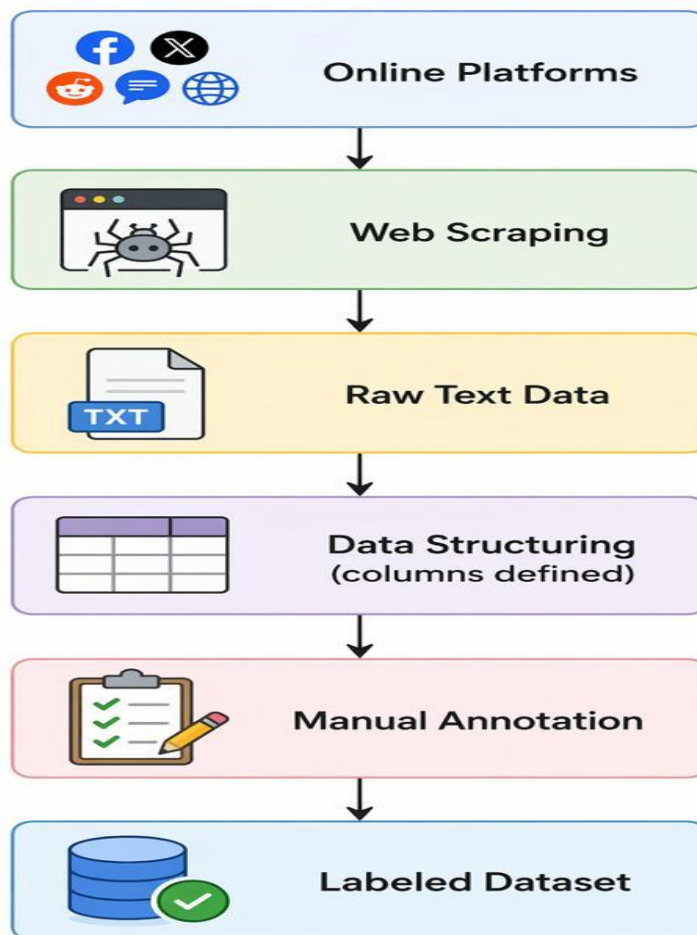


Figure 11: Data collection workflow diagram

2.4.2 Data Preprocessing

Raw data collected through web scraping is unstructured and cannot be directly used for machine learning. Therefore, a systematic preprocessing pipeline is designed to transform raw text into a consistent and analyzable format. The preprocessing is applied **sequentially**, meaning the output of one step becomes the input of the next.

Step 1: Lowercasing (Text Normalization)

Lowercasing is applied as the first stage of the preprocessing pipeline to ensure uniform text representation. In this process, each message is read from the dataset as a raw text string, after which the system traverses each character and converts all uppercase letters into their lowercase equivalents before storing the modified text. This transformation standardizes the textual data by eliminating case-based variations, thereby preventing duplication of features and ensuring that words such as “Hate” and “hate” are treated as identical tokens during analysis. This step is essential in natural language processing as it reduces dimensionality and improves the consistency of feature extraction [26].

Step 2: Removal of Special Characters

The removal of special characters is performed to eliminate non-alphabetic elements that do not contribute to the semantic meaning of the text. In this process, each message is scanned character by character, retaining only alphabetic characters (a–z, A–Z) and spaces, while punctuation marks, digits, and other symbols are removed and replaced with empty spaces. This filtering is guided by a rule-based condition where a character is preserved only if it satisfies the constraint $c \in [a-zA-Z]$. This step ensures that the resulting text is free from noise and is composed solely of meaningful linguistic components, thereby improving the quality of subsequent text processing and feature extraction [29].

Step 3: Removal of URLs

The removal of URLs is carried out to eliminate web links that do not contribute to the semantic meaning of the text. In this process, each message is scanned to identify patterns corresponding to web addresses, particularly substrings beginning with “http”, “https”, or “www”. Once detected, the complete link string is extracted and removed entirely from the text. This step is necessary because URLs introduce noise into the dataset and do not provide useful linguistic

information for classification tasks, thereby negatively affecting the quality of feature extraction and model performance [26].

Step 4: Removal of Mentions

The removal of user mentions is performed to eliminate platform-specific references that do not contribute to the semantic interpretation of the text. In this process, each message is analyzed using pattern recognition to detect tokens that begin with the “@” symbol, which typically represent usernames. Once identified, the complete mention (e.g., @username) is extracted and removed from the text. This step helps reduce noise and prevents the model from learning irrelevant patterns associated with specific user identifiers, thereby improving the generalization capability of the system [26].

Step 5: Stopword Removal

Stopword removal is performed to eliminate commonly used words that carry little semantic value in text analysis. In this process, each message is first split into individual words, after which each word is compared against a predefined stopwords list containing frequently occurring terms such as “is”, “the”, and “and”. Words that appear in this list are removed, while only meaningful and informative words are retained. The filtering follows a simple logical condition: if a word belongs to the stopwords set, it is discarded; otherwise, it is preserved. This step reduces noise, decreases dimensionality, and enhances the effectiveness of feature extraction for machine learning models [30].

Step 6: Tokenization

Tokenization is performed to convert the cleaned text into a structured format by breaking it into smaller units known as tokens. In this process, the text is analyzed to identify spaces as natural separators between words, after which the sentence is split into individual word units. Each resulting word is then stored in a list-like structure, forming a sequence of tokens that can be further processed in subsequent stages. This step is essential for enabling operations such as stopwords removal and stemming, as well as preparing the text for numerical representation in machine learning models [30].

Step 7: Stemming

Stemming is applied to reduce words to their root or base forms using a rule-based process. In this step, each token is examined individually to identify common suffix patterns such as “ing”, “ed”, and “ly”, which are then removed according to predefined linguistic rules. The resulting word is replaced with its root form, for example transforming “running” into “run” and “hated” into “hate”. This process helps normalize variations of the same word, reduces dimensionality, and improves the consistency of feature representation in the dataset. The approach is based on the stemming algorithm proposed by Porter (1980) [31].

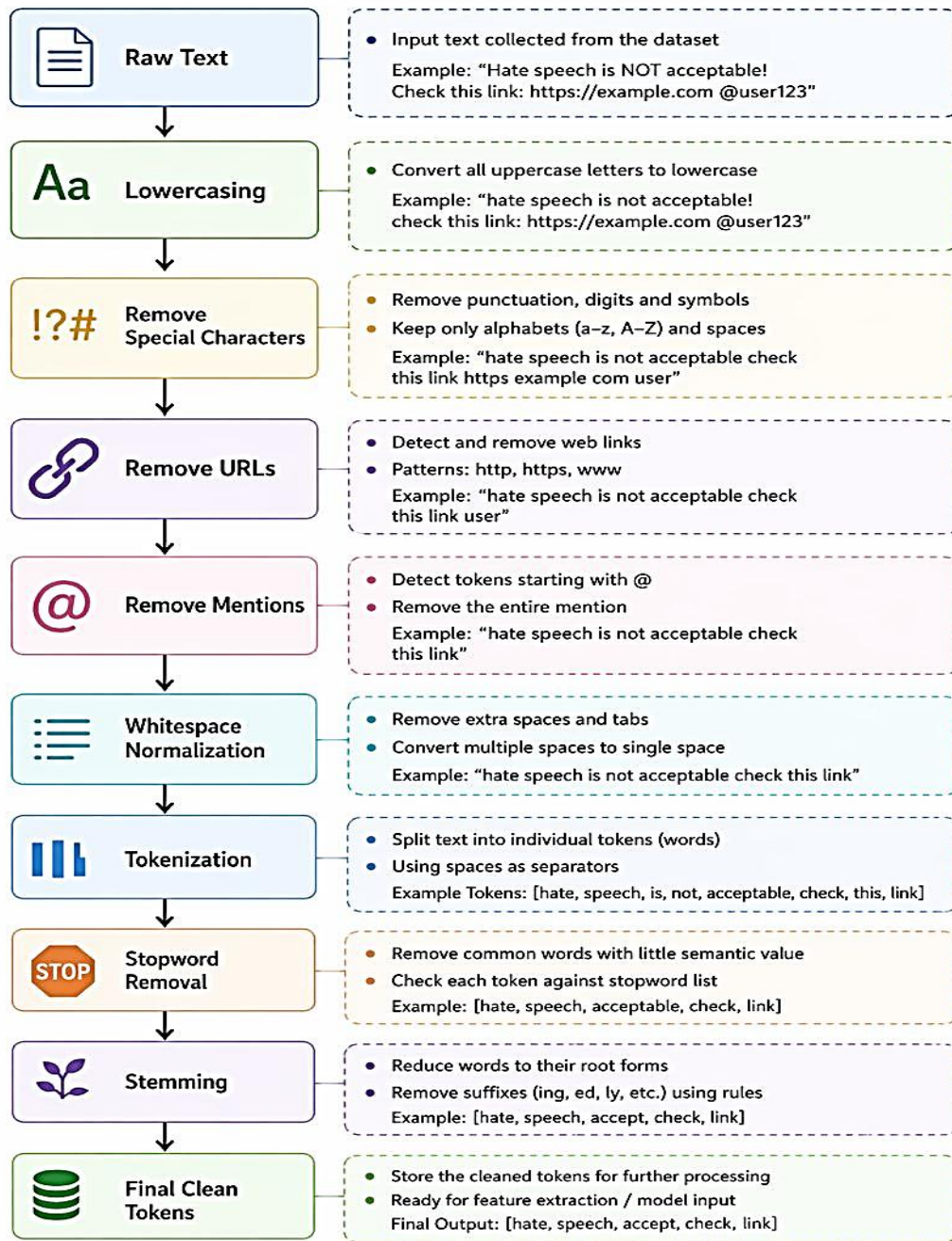


Figure 12: Data Preprocessing workflow diagram

2.4.2.1 Preprocessing Example:

Input: "You are SO stupid!!! <http://bad.com> @john"

Step 1: Lowercasing [str.lower()]: "you are so stupid!!! <http://bad.com> @john"

Step 2: Remove special chars [re.sub()]: "you are so stupid httpbadcom john"

Step 3: Remove URLs [re.sub()]: "you are so stupid john"

Step 4: Remove mentions [re.sub()]: "you are so stupid"

Step 5: Remove stopwords [NLTK]: "stupid"

Step 6: Tokenization [NLTK word_tokenize()]: ["stupid"]

Step 7: Stemming [NLTK PorterStemmer]: ["stupid"]

Final output: "stupid"

Pseudocode

Algorithm 2: Text Preprocessing

Input: Raw text string raw_text

Output: List of preprocessed tokens tokens

1. $\text{text} \leftarrow \text{raw_text.lower()}$
2. $\text{text} \leftarrow \text{re.sub}(\text{r}'[\text{^a-zA-Z}\text{s}]', '', \text{text})$
3. $\text{text} \leftarrow \text{re.sub}(\text{r}'\text{http}\text{S}+', '', \text{text})$
4. $\text{text} \leftarrow \text{re.sub}(\text{r}'@\text{w}+', '', \text{text})$
5. $\text{stop_words} \leftarrow \text{set}(\text{stopwords.words}('english'))$
6. $\text{words} \leftarrow \text{text.split}()$
7. $\text{filtered} \leftarrow [\text{word for word in words if word not in stop_words}]$
8. $\text{stemmer} \leftarrow \text{PorterStemmer}()$
9. $\text{tokens} \leftarrow [\text{stemmer.stem}(\text{word}) \text{ for word in filtered}]$
10. **Return tokens** [Bird et al., 2009; Porter, 1980]

2.4.2.2 Handling Missing Values (Imputation)

Real-world datasets collected through web scraping often contain missing, incomplete, or inconsistent entries due to extraction errors, deleted content, or unavailable metadata. To maintain dataset integrity and ensure compatibility with machine learning algorithms, a systematic imputation strategy is applied. Missing textual fields are replaced with an empty string to preserve the structural consistency of the dataset without introducing artificial semantic

noise, while missing numerical values are replaced with zero to maintain mathematical validity during feature processing and model training. This approach ensures that vectorization techniques such as TF-IDF can be applied without encountering null-related errors, and that machine learning models can process the dataset without interruption. The strategy follows standard data preprocessing practices commonly adopted in text mining and machine learning workflows [22].

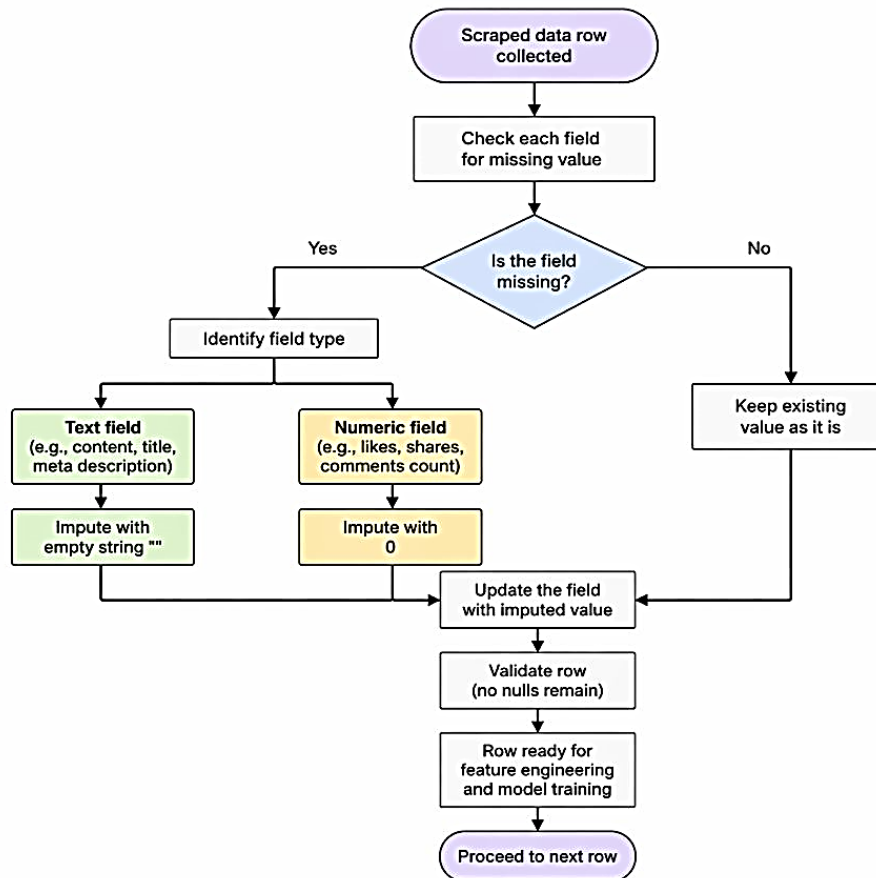


Figure 13: Missing Value Handling Process in Scraped Dataset

2.4.3 Feature Extraction with TF-IDF

Machine learning models cannot understand raw text. Feature extraction converts preprocessed text into numerical vectors that algorithms can process. This project uses **TF-IDF (Term Frequency-Inverse Document Frequency)** to transform text data into numerical features [9][33].

2.4.3.1 Mathematical Formulation [33]

Term Frequency (TF) measures how often a word appears in a message.

$$TF(t, d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}}$$

Where :

- ❖ $f_{t,d}$ = raw count of term t in document d
- ❖ $\sum_{t' \in d} f_{t',d}$ = total number of terms in document d

Inverse Document Frequency (IDF) measures how rare or common a word is across all messages.

$$IDF(t, D) = \log \left(\frac{|D|}{|\{d \in D: t \in d\}|} \right)$$

Where:

- $|D|$ = total number of documents in the corpus
- $|\{d \in D: t \in d\}|$ = number of documents containing term t

The TF-IDF Score combines both to identify important words.

$$TF - IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

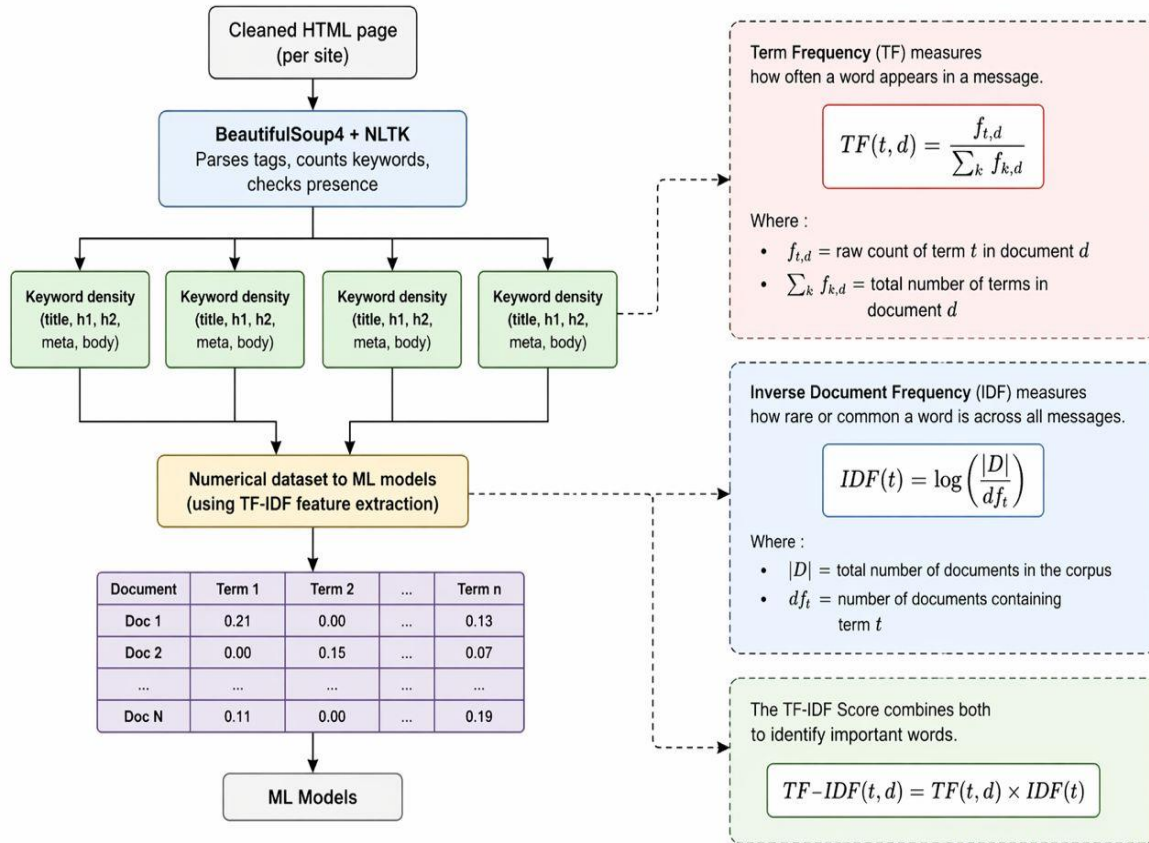


Figure 14: Feature Extraction with TF-IDF workflow

2.4.3.2 Worked Example

Consider a corpus of $|D| = 1000$ documents. The word "kill" appears in document A three times. Document A has 50 total words. The word "kill" appears in 10 documents total.

Step 1: Calculate TF

$$TF("kill", A) = \frac{3}{50} = 0.06$$

Step 2: Calculate IDF

$$IDF("kill", D) = \log\left(\frac{1000}{10}\right) = \log(100) = 2.0$$

Step 3: Calculate TF-IDF

$$TF-IDF("kill", A, D) = 0.06 \times 2.0 = 0.12$$

Interpretation: A high TF-IDF score indicates a word that is frequent in the current document but rare across all documents exactly the type of word that distinguishes hate speech (e.g., slurs) from normal conversation.

2.4.3.3 Vectorization Parameters [9]

Table 13: TF-IDF Vectorization Parameters

Parameter	Value	Explanation
max_features	5,000	Use the 5,000 most informative terms
ngram_range	(1, 2)	Unigrams + bigrams (e.g., "kill", "kill you")
min_df	2	Ignore terms appearing in fewer than 2 documents
max_df	0.95	Ignore terms appearing in more than 95% of documents (too common)
sublinear_tf	True	Apply $1+\log(\text{tf})$ scaling

2.4.3.4 TF-IDF Vectorization Pseudocode

Algorithm: TF-IDF Vectorization

Input: Preprocessed token list tokens, fitted vectorizer V

Output: Feature vector $x \in \mathbb{R}^{5000}$

1. Input: List of preprocessed tokens (comments)
2. Output: TF-IDF feature matrix X ($n_{\text{comments}} \times 5,000$)
 - a. $\text{vectorizer} \leftarrow \text{TfidfVectorizer}(\text{max_features} = 5000,$
 4. $\text{ngram_range} = (1,2),$
 5. $\text{min_df} = 2,$
 6. $\text{max_df} = 0.95,$
 7. $\text{sublinear_tf} = \text{True}$
 8. $)$
 - a. $\text{comments_as_strings} \leftarrow [" ".join(\text{tokens}) \text{ for tokens in comments}]$
 - b. $X \leftarrow \text{vectorizer.fit_transform}(\text{comments_as_strings})$

- c. $X_dense \leftarrow X.toarray()$
- d. $vocabulary \leftarrow vectorizer.get_feature_names_out()$

9. Return X_dense , $vectorizer$, $vocabulary$

2.4.4 Model Selection (Classification)

The classification phase is designed to remove human subjectivity from hate speech detection by utilizing a supervised machine learning model to automatically categorize text into predefined classes (i.e., “Hate Speech” or “Normal Speech”).

2.4.4.1 Objective and Dataset Construction

The primary objective of the classification model is to evaluate the numerical features extracted in the previous phase (TF-IDF representations of text data) and assign each input text a discrete label indicating whether it contains hate speech or not. Since this is a supervised learning problem, the model requires a reliable labelled dataset to learn meaningful classification boundaries [1].

To reduce the cost and bias associated with manual annotation, this project relies on publicly available benchmark datasets widely used in hate speech detection research, particularly social media corpora such as Twitter datasets and other annotated text collections [2], [3].

The dataset is composed of two main categories:

- ❖ **Hate Speech:** Text containing abusive, offensive, or discriminatory language targeting individuals or groups
- ❖ **Normal Speech:** Neutral or non-offensive content without harmful intent

Because real-world text datasets are typically imbalanced (with a higher proportion of normal speech), data balancing techniques are applied during preprocessing. These include:

- ❖ **Class weight adjustment**
- ❖ **SMOTE (Synthetic Minority Over-sampling Technique) [4]**
- ❖ **Undersampling of majority class (when necessary)**

These methods ensure that the classifier does not become biased toward the majority class and maintains strong detection performance for minority hate speech instances [4].

2.4.4.2 Algorithm Selection: Support Vector Machine (SVM)

Based on the nature of the problem high-dimensional sparse text data represented using TF-IDF vectors the **Support Vector Machine (SVM)** was selected as the core classification algorithm for this system [32], [36].

2.4.4.3 Selection Rationale from Literature

SVM has been extensively validated in literature as one of the most effective algorithms for text classification tasks:

- ❖ Joachims (1998) demonstrated that SVM significantly outperforms Naïve Bayes and Decision Trees in text categorization due to its ability to handle high-dimensional feature spaces effectively [36].
- ❖ Wang et al. (2017) showed that SVM achieves higher F1-scores compared to Logistic Regression and Random Forest in hate speech detection tasks on social media datasets [37].
- ❖ Fortuna and Nunes (2018) further confirmed that linear SVM remains a strong baseline model for abusive language detection due to its robustness and generalization ability [38].

From these studies, it is evident that SVM consistently performs well in NLP classification problems, especially when dealing with sparse representations such as TF-IDF.

SVM is a supervised learning algorithm that finds the optimal hyperplane which separates data points of different classes while maximizing the margin between them. The margin refers to the distance between the decision boundary and the closest data points, known as support vectors [32].

2.4.4.4 Linear Decision Function

For text classification using TF-IDF features, a linear kernel is typically used due to its efficiency in high-dimensional spaces. The decision function is defined as:

$$f(x) = w \cdot x + b$$

Where:

- ❖ w = weight vector
- ❖ x = TF-IDF feature vector
- ❖ b = bias term

The classification rule is:

- ❖ If $f(x) \geq 0 \rightarrow$ Hate Speech
- ❖ If $f(x) < 0 \rightarrow$ Normal Speech

This formulation is widely used in linear SVM models for NLP classification tasks [36], [37].

2.4.4.5 Margin Maximization Principle

The core objective of SVM is to maximize the margin between classes, which improves generalization and reduces overfitting. This can be conceptually expressed as:

maximize margin between classes in feature space

Maximizing this margin ensures that the classifier performs well on unseen data, which is critical in noisy and unstructured text environments such as social media datasets [37].

2.4.4.6 Handling High-Dimensional Text Data

SVM is particularly well-suited for this project because TF-IDF representations produce high-dimensional sparse feature spaces. According to Joachims (1998), SVM performs exceptionally well in such environments due to its reliance on support vectors rather than the entire dataset [36].

Additionally, linear SVM avoids the computational complexity of non-linear transformations while still achieving strong classification performance in NLP applications [38].

2.4.4.7 Final Model Output

The trained SVM model performs binary classification as follows:

$$Y = f(x) \in \{\text{Hate Speech}, \text{Normal Speech}\}$$

Where:

- ❖ x represents the TF-IDF feature vector
- ❖ Y represents the predicted class label

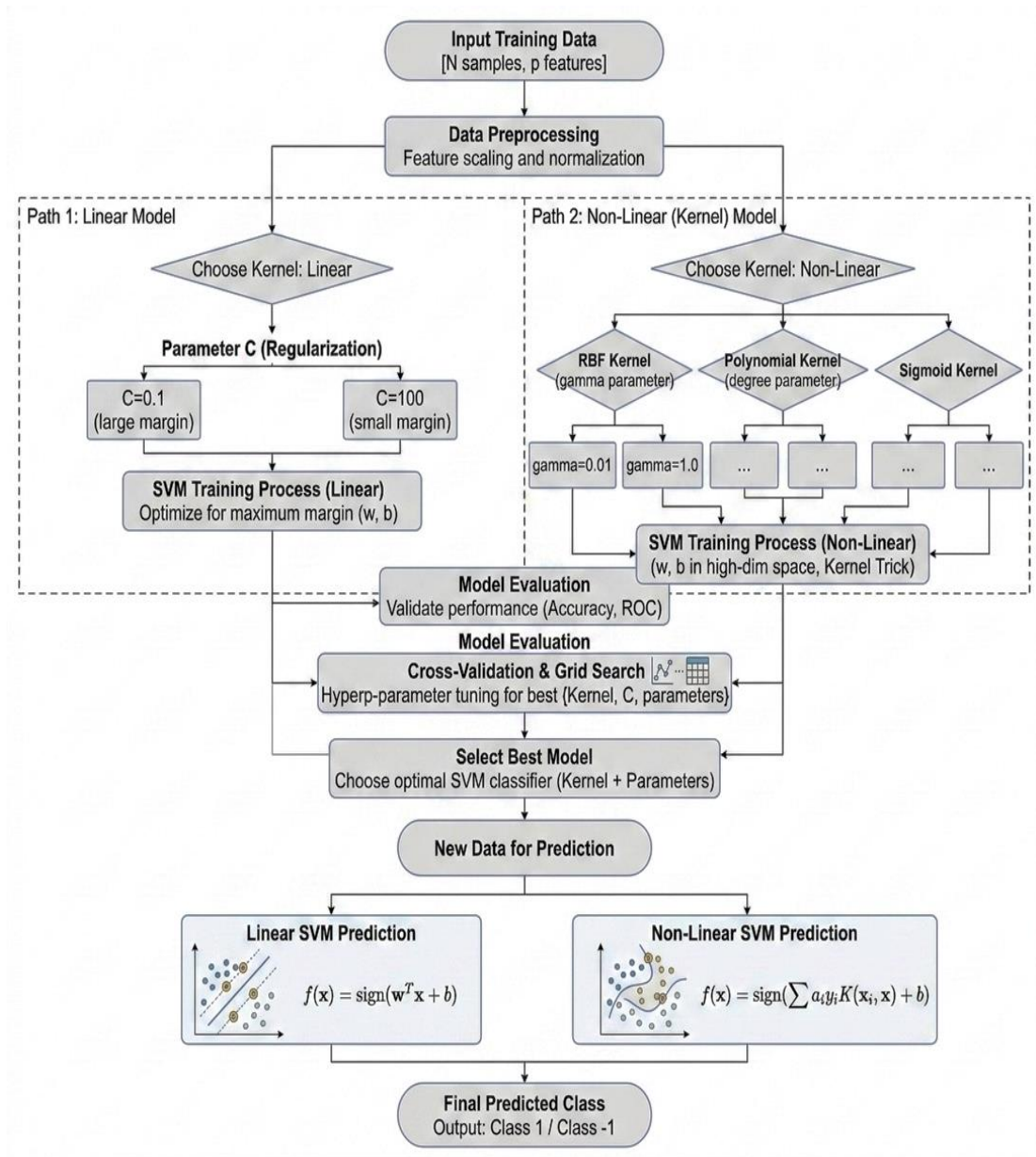


Figure 15:: Support vector machine workflow

2.4.5 Model Training

Model training is the stage where the selected Support Vector Machine (SVM) algorithm learns patterns from the labeled data [2]. The model learns to map TF-IDF feature vectors to binary labels (Hate Speech = 1, Normal = 0) using the training data [9].

What happens during training:

- ❖ The model receives input features (TF-IDF vectors) and known labels
- ❖ SVM finds the optimal hyperplane that separates hate speech from normal comments
- ❖ After training, the model can predict labels for new, unseen comments

2.4.5.1 Data Splitting Strategy

The dataset is divided into training and testing sets to evaluate performance on unseen data [9].

Table 14:: Data Split Distribution

Split	Percentage	Number of Comments (based on 10,000 total)	Purpose
Training	80%	8,000 comments	Teach the model patterns
Testing	20%	2,000 comments	Evaluate performance on unseen data

$$\text{Train_size} = \text{Total} \times 0.80$$

$$\text{Test_size} = \text{Total} \times 0.20$$

2.4.5.2 Handling Class Imbalance

Hate speech is rare compared to normal content. Without adjustment, the model would become biased toward predicting "Normal" for all inputs [2]. This project uses class weighting to address imbalance.

Balanced Class Weight Formula [9]:

$$\omega_c = \frac{N}{k \times N_c}$$

Where:

- ❖ N = total number of comments
- ❖ k = number of classes (2)
- ❖ N_c = number of comments in class c

2.4.5.3 Cross-Validation

To ensure the model generalizes well, 5-fold cross-validation is used during training [9].

Procedure:

1. Split training data into $k=5$ equal folds
2. For each fold $i=1$ to k :
 - a. Train on all folds except fold i
 - b. Validate on fold i
 - c. Record performance metric M_i
3. Final performance = average of all k folds:

$$M_{cv} = \frac{1}{k} \sum_{i=1}^k M_i$$

Standard Deviation:

$$\sigma_{cv} = \sqrt{\frac{1}{k-1} \sum_{i=1}^k (M_i - M_{cv})^2}$$

Low standard deviation indicates stable performance across folds.

CROSS VALIDATION, EXPLAINED

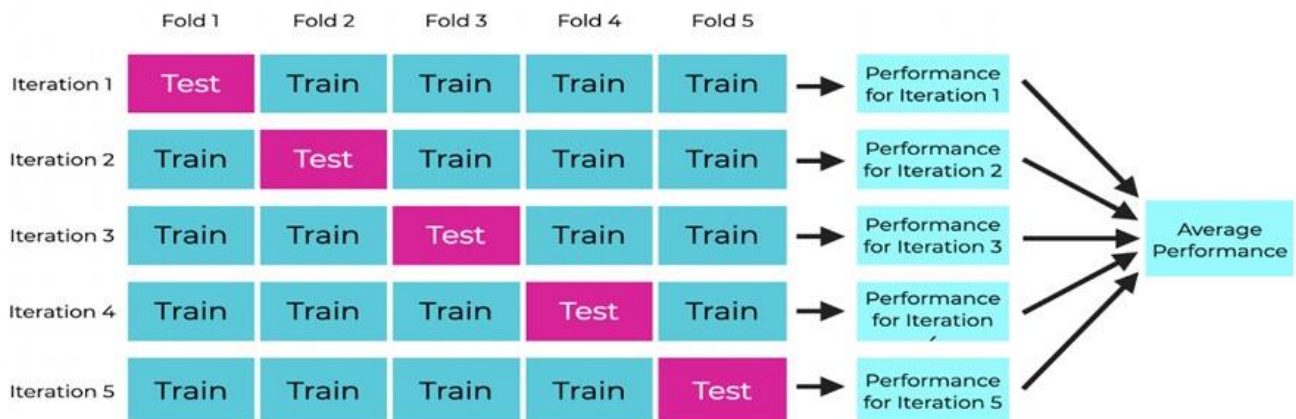


Figure 16: Cross validation

Training Configuration

Table 15: SVM Training Configuration

Kernel	'linear'	Best for text data with TF-IDF
C	1.0	Regularization parameter
class_weight	'balanced'	Handles class imbalance [2]
random_state	42	Ensures reproducibility
max_iter	1000	Maximum iterations for convergence

Training Pseudocode

Algorithm: Model Training [2][9]

Input: TF-IDF matrix X, labels y

Output: Trained SVM model

1. X_train, X_test, y_train, y_test $\leftarrow$ train_test_split(
2. X, y, test_size=0.20, random_state=42, stratify=y
3.)
4. model $\leftarrow$ LinearSVC(
5. C = 1.0,
6. kernel = 'linear',
7. class_weight = 'balanced',
8. random_state = 42,
9. max_iter = 1000
10.)
11. cv_scores $\leftarrow$ cross_val_score(
12. model, X_train, y_train, cv=5, scoring='f1_macro'
13.)
14. model.fit(X_train, y_train)

15. $y_pred \leftarrow \text{model.predict}(X_test)$
16. Return model, y_pred , y_test , cv_scores

Example

Before training, a comment like "kill all Muslims" is converted to a TF-IDF vector [0.044, 0, 0.051, ...] with label 1 (Hate Speech). A normal comment like "good morning everyone" becomes a different vector [0, 0.1, 0, ...] with label 0 (Normal).

During training, SVM learns the boundary that separates these two types of vectors. It adjusts its hyperplane to maximize the distance between hate speech and normal comments.

After training, when a new comment "kill all Muslims" is received, the model converts it to a TF-IDF vector and checks which side of the hyperplane it falls on, then predicts 1 (Hate Speech).

2.4.6 Model Evaluation

Model evaluation measures how well the trained SVM model performs on unseen data (the test set) [2]. This project uses four standard metrics: accuracy, precision, recall, and F1-score [7]. The macro F1-score is the primary metric because it balances precision and recall for both classes equally.

Confusion Matrix

The confusion matrix shows the model's predictions compared to actual labels [2].

Table 16: Confusion Matrix Structure (2-Class)

Actual \ Predicted	Normal (0)	Hate Speech (1)
Normal (0)	True Negative (TN)	False Positive (FP)
Hate Speech (1)	False Negative (FN)	True Positive (TP)

Mathematical Formulations [7]

Accuracy: Overall correctness

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision: Of all flagged hate speech, how many were correct

$$\text{Precision} = \frac{TP}{TP + FP}$$

Recall: Of all actual hate speech, how many were caught

$$\text{Recall} = \frac{TP}{TP + FN}$$

F1-Score: Harmonic mean of precision and recall (Primary Metric)

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Macro F1-Score: Average of F1 scores for both classes

$$F1_{\text{macro}} = \frac{F1_{\text{normal}} + F1_{\text{hate}}}{2}$$

Weighted F1-Score (Accounts for imbalance):

$$F1_{\text{weighted}} = \sum_c \left(\frac{N_c}{N} \times F1_c \right)$$

Where N_c = number of instances of class c , N = total instances.

Example Calculation

Suppose the model makes predictions on 2,000 test comments. The actual breakdown is 1,800 normal comments and 200 hate speech comments.

After comparing predictions to actual labels:

The model correctly identified 1,692 normal comments (TN)

It incorrectly flagged 108 normal comments as hate speech (FP)

It correctly identified 144 hate speech comments (TP)

It missed 56 hate speech comments (FN)

From these numbers:

$$\text{Accuracy} = \frac{(1,692 + 144)}{2,000} = 0.918 \text{ (91.8\%)}$$

$$\text{Precision} = \frac{144}{(144 + 108)} = 0.571 \text{ (57.1\%)}$$

$$\text{Recall} = \frac{144}{(144 + 56)} = 0.720 \text{ (72.0\%)}$$

$$F1\text{-Score} = 2 \times \frac{(0.571 \times 0.720)}{(0.571 + 0.720)} = 0.637 \text{ (63.7\%)}$$

The macro F1-score averages the F1 for both classes, giving equal weight to hate speech detection despite having fewer samples.

Table 17: Evaluation Metrics

Metric	Formula	Best Value	Purpose
Accuracy	$(TP+TN)/(Total)$	1.0	Overall correctness
Precision	$TP/(TP+FP)$	1.0	Exactness of hate speech detection
Recall	$TP/(TP+FN)$	1.0	Completeness of hate speech detection
F1-Score	$2 \times (P \times R) / (P + R)$	1.0	Balance of precision and reca

Pseudocode

Algorithm 5: Model Evaluation [2][7]

Input: True labels y_true , Predicted labels y_pred

Output: Performance metrics (Accuracy, Precision, Recall, F1)

```

1.  $TP \leftarrow \text{sum}((y\_true == 1) \& (y\_pred == 1))$ 
2.  $TN \leftarrow \text{sum}((y\_true == 0) \& (y\_pred == 0))$ 
3.  $FP \leftarrow \text{sum}((y\_true == 0) \& (y\_pred == 1))$ 
4.  $FN \leftarrow \text{sum}((y\_true == 1) \& (y\_pred == 0))$ 
5.  $\text{accuracy} \leftarrow (TP + TN) / (TP + TN + FP + FN)$ 
6. IF  $(TP + FP) > 0$ :
     $\text{precision} \leftarrow TP / (TP + FP)$ 
ELSE:
     $\text{precision} \leftarrow 0$ 
7. IF  $(TP + FN) > 0$ :
     $\text{recall} \leftarrow TP / (TP + FN)$ 
ELSE:
     $\text{recall} \leftarrow 0$ 
8. IF  $(\text{precision} + \text{recall}) > 0$ :
     $f1 \leftarrow 2 * (\text{precision} * \text{recall}) / (\text{precision} + \text{recall})$ 

```

ELSE:

$f1 \leftarrow 0$

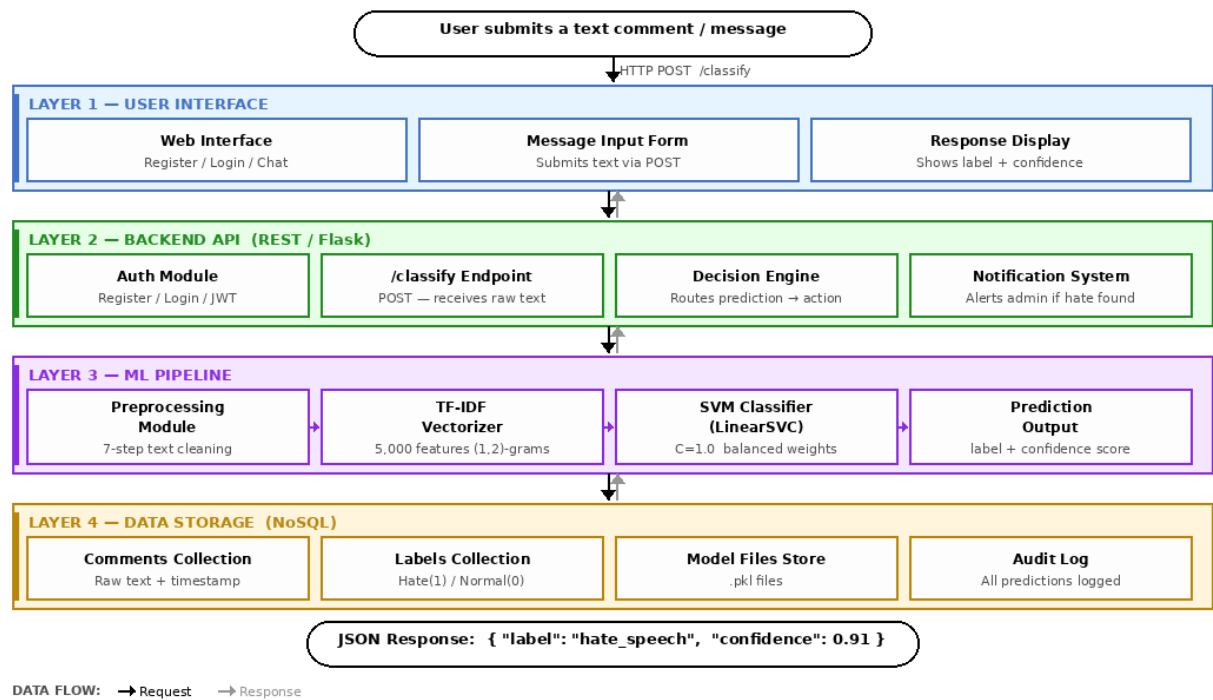
9. Print("Accuracy:", accuracy)
10. Print("Precision:", precision)
11. Print("Recall:", recall)
12. Print("F1-Score:", f1)
13. Return accuracy, precision, recall, f1

2.5 SYSTEM ARCHITECTURE

The system architecture follows a client-server model with distinct components responsible for specific functions. This modular architecture ensures that each component can be developed, tested, and maintained independently while working together seamlessly to provide real-time content moderation.

Table 18: System component description

Component	Description
User Interface	Web interface where users register, log in, and post messages
Backend API	Handles requests, authentication, and communication between components
Preprocessing Module	Cleans and prepares text for analysis
Feature Extraction Module	Converts text into numerical features using TF-IDF
Machine Learning Model	Classifier that detects hate speech and harassment
Decision Engine	Determines automated actions based on prediction and severity
Notification System	Sends alerts to administrators and notifications to users



2.6 API Design

API (Application Programming Interface) design defines how the trained machine learning model will communicate with external applications. Since this project focuses on model development and evaluation, the API serves as a bridge for deploying the model so that other systems can send text comments and receive hate speech predictions.

2.6.1 API Architecture

The system uses a **RESTful API** architecture, which is the standard for web-based machine learning model deployment. REST APIs use HTTP methods and return data in JSON format.

Key characteristics:

- ❖ **Stateless:** Each request contains all information needed
- ❖ **JSON format:** Lightweight and easy to parse
- ❖ **HTTP methods:** POST for predictions

2.6.2 Core API Endpoint

HTTP Method	Endpoint	Description	Input	Output
POST	/predict	Send text for hate speech detection	JSON with "text" field	JSON with prediction and confidence

2.4.8.1 API Processing Steps

Step	Action	Description
1	Receive input	Extract "text" field from JSON request
2	Preprocess	Apply same 7-step pipeline used in training
3	Vectorize	Convert clean text to TF-IDF using saved vectorizer
4	Predict	Load trained SVM model and predict class
5	Format response	Create JSON with prediction, label, confidence
6	Return	Send response back to client

2.7 Database Design

The data storage layer is the central repository for the **Hate Speech Detection System**. It must manage the transition from raw, unstructured data captured during web scraping to refined, labeled datasets used for machine learning. Given the deeply irregular nature of scraped HTML and the JSON-based architecture of the Python ML pipeline, a **Document-Oriented NoSQL database** was selected.

2.7.1 The Nature of Document-Oriented NoSQL

A Document-Oriented NoSQL database stores information in flexible, JSON-like documents rather than fixed tables and rows. In this paradigm, a "document" is a self-contained unit that can house diverse data types, including nested arrays for metadata and sub-documents for model scores.

Data is stored as **BSON (Binary JSON)**, which allows the system to natively handle complex types like timestamps for when a post was scraped and binary flags for moderation status. For this project, a single entry in the database represents a complete "Audit Unit" containing the

raw scraped text, source URL, extracted linguistic features, and the final hate speech classification all stored within one retrievable object.

2.7.2 Why NoSQL is Essential for Scraped ML Data

Using web scraping to build a database introduces high levels of data "noise" and structural variability that traditional databases cannot handle efficiently.

- ❖ **Handling Scraped Irregularity:** Web scraping is inherently unpredictable. One website might store comments in <div> tags with "author" metadata, while another uses tags with no author information. In a relational (SQL) database, this would require a rigid schema that breaks whenever the scraper encounters a new HTML structure. NoSQL handles this effortlessly; the database simply accepts whatever fields the scraper successfully extracts without requiring a predefined template.
- ❖ **Direct Pipeline Integration:** The system is built using a Python-based ML pipeline. Both the web scraper (BeautifulSoup/Scrapy) and the ML models (Scikit-learn/TensorFlow) interact with data as Python dictionaries. Since NoSQL documents are effectively JSON, data can flow from the scraper to the database, and then to the ML model, without any expensive data conversion or "translation" layers.
- ❖ **Optimizing for Training and Inference:** Building a custom database requires frequent updates as more data is scraped. The NoSQL "Schema-on-Read" approach allows the system to evolve; if we decide to start scraping a new feature (like "user-likes" or "time-of-post"), we can begin storing that data immediately without migrating the entire existing database. This flexibility is vital for maintaining the speed required to provide a **human-in-the-loop** moderator with real-time detection results.

CONCLUSION

This chapter presented the complete design of the machine learning pipeline for hate speech detection. The pipeline consists of six stages: Data Collection, Data Preprocessing, Feature Extraction, Model Selection, Model Training, and Model Evaluation.

The pipeline is designed to be reproducible, with each stage documented through workflow diagrams, examples, and references. The API design and database design provide a foundation for future deployment, allowing external applications to send text and receive hate speech predictions in real time [9].

REFERENCES

- [1] Alharbi, A. et al. (2023) 'Hate speech detection using machine learning: A survey', *International Journal of Advanced Computer Science and Applications*.
- [2] Davidson, T., Warmley, D., Macy, M. and Weber, I. (2017) 'Automated Hate Speech Detection and the Problem of Offensive Language', *Proceedings of the 11th International AAAI Conference on Web and Social Media (ICWSM '17)*, Montreal, Canada, pp. 512-515.
- [3] Bashir, I.S. and Ahmad, M.A. (2026) 'Hate speech detection using machine learning techniques', *FUDMA Journal of Sciences*.
- [4] Jain, A. and Sharma, S. (2023) 'Hate speech detection based on word embedding and linguistic features', *Indian Journal of Science and Technology*.
- [5] 'Text preprocessing techniques for effective hate speech detection' (2023) *Science World Journal*.
- [6] Natha, A., Behera, B., Mishra, D., Sahu, S.R. and Mohapatra, S. (2024) 'A Comparative Analysis of Machine Learning Models Used for Hate Speech (HS) Detection of Odia Language', *International Journal of Intelligent Systems and Applications in Engineering*, 12(18), pp. 1-13.
- [7] Das, S. (2023) 'Performance analysis of machine learning algorithms on hate speech detection from Twitter', ResearchGate.
- [8] Saroj, A. and Pal, S. (2020) 'An Indian language social media collection for hate and offensive speech', *Proceedings of the Workshop on Resources and Techniques for User and Author Profiling in Abusive Language*, Marseille, France, pp. 2-8.
- [9] Pedregosa, F. et al. (2011) 'Scikit-learn: Machine Learning in Python', *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830.
- [10] Breiman, L. (2001) 'Random Forests', *Machine Learning*, 45(1), pp. 5-32.
- [11] Waseem, Z. and Hovy, D. (2016) 'Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter', *Proceedings of NAACL Student Research Workshop*, pp. 88-93.

- [12] Burnap, P. and Williams, M.L. (2015) 'Cyber hate speech on Twitter: An application of machine classification and statistical modeling for policy and decision making', *Policy & Internet*, 7(2), pp. 223-242.
- [13] Nobata, C. et al. (2016) 'Abusive Language Detection in Online User Content', *Proceedings of the 25th International Conference on World Wide Web*, pp. 145-153.
- [14] Poletto, F. et al. (2021) 'Resources and benchmark corpora for hate speech detection: A systematic review', *Language Resources and Evaluation*, 55, pp. 477-523.
- [15] Mollas, I., Chrysopoulou, Z., Karlos, S. and Tsoumakas, G. (2022) 'ETHOS: a multi-label hate speech detection dataset', *Complex & Intelligent Systems*.
- [16] Devlin, J., Chang, M.W., Lee, K. and Toutanova, K. (2019) 'BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding', *Proceedings of NAACL-HLT 2019, Minneapolis*, pp. 4171–4186.
- [17] Conneau, A., Khandelwal, K., Goyal, N., Chaudhary, V., Wenzek, G., Guzmán, F., Grave, E., Ott, M., Zettlemoyer, L. and Stoyanov, V. (2020) 'Unsupervised Cross-lingual Representation Learning at Scale', *Proceedings of ACL 2020*, pp. 8440–8451.
- [18] Hochreiter, S. and Schmidhuber, J. (1997) 'Long Short-Term Memory', *Neural Computation*, 9(8), pp. 1735–1780.
- [19] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is All You Need', *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*.
- [20] Waseem, Z. and Hovy, D. (2016) 'Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter', *Proceedings of NAACL Student Research Workshop*, pp. 88–93.
- Natha, A., Behera, B., Mishra, D., Sahu, S.R. and Mohapatra, S. (2024)** 'A Comparative Analysis of Machine Learning Models Used for Hate Speech (HS) Detection of Odia Language', *International Journal of Intelligent Systems and Applications in Engineering*, 12(18), pp. 1-13.
- [21] Cohen, J. (1960). *A Coefficient of Agreement for Nominal Scales*. *Educational and Psychological Measurement*, 20(1), 37–46.

- [22] Han, J., Kamber, M., & Pei, J. (2012). *Data Mining: Concepts and Techniques* (3rd ed.). Morgan Kaufmann.
- [8] Saroj, A.; Pal, S. An Indian language social media collection for hate and offensive speech. In *Proceedings of the Workshop on Resources and Techniques for User and Author Profiling in Abusive Language*, Marseille, France, 11–16 May 2020; pp. 2–8.
- [16] Mishra, B. K., & Sahoo, R. (2018). A hybrid knowledge mining approach to develop a system framework for Odia language text processing. *Materials Today: Proceedings*, 5(1), 1335-1340.
- [23] Jena, M. K., & Mohanty, S. (2020). Predicting the Impact of Odia Newspaper Articles on Public Opinion. In *Progress in Computing, Analytics and Networking: Proceedings of ICCAN 2019* (pp. 265-272). Springer Singapore.
- [25] Fauzi, M.A.; Yuniarti, A. Ensemble method for Indonesian Twitter hate speech detection. *Indones. J. Electr. Eng. Comput. Sci.* 2018, 11, 294–299. [CrossRef]
- [26] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [28] Pereira-Kohatsu, J.C.; Quijano-Sánchez, L.; Liberatore, F.; Camacho-Collados, M. Detecting and monitoring hate speech in Twitter. *Sensors* 2019, 19, 4654. [CrossRef]
- [29] Sahoo, Rekhanjali & Mishra, Brojo & Das, Bishwa. (2022). Odia Text Classification Using Naïve Bayes Algorithm: An Empirical Study. *ECS Transactions*. 107. 8175-8180. 10.1149/10701.8175ecst. [30] Poletto, F.; Basile, V.; Sanguinetti, M.; Bosco, C.; Patti, V. Resources and benchmark corpora for hate speech detection: A systematic review. *Lang. Resour. Eval.* 2021, 55, 477–523. [Google Scholar] [CrossRef]
- [29] Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- [30] Jurafsky, D., & Martin, J. H. (2009). *Speech and Language Processing*. Pearson.
- [31] Porter, M. F. (1980). An Algorithm for Suffix Stripping. *Program*, 14(3), 130–137.
- [32] Cortes, C., & Vapnik, V. (1995). *Support-vector networks*. *Machine Learning*, 20(3), 273–297.

- [33] Davidson, T. et al. (2017). *Automated Hate Speech Detection and the Problem of Offensive Language*. ICWSM.
- [34] Waseem, Z., & Hovy, D. (2016). *Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter*. NAACL.
- [35] Chawla, N. V. et al. (2002). *SMOTE: Synthetic Minority Over-sampling Technique*. Journal of Artificial Intelligence Research.
- [36] Joachims, T. (1998). *Text categorization with Support Vector Machines*. ECML.
- [37] Wang, Y. et al. (2017). *Hate Speech Detection using SVM and Machine Learning Techniques*. IEEE Access.
- [38] Fortuna, P., & Nunes, S. (2018). *A Survey on Automatic Detection of Hate Speech in Text*. ACM Computing Surveys.
- [39] Sachdeva, J.; Chaudhary, K.K.; Madaan, H.; Meel, P. Text-Based Hate-Speech Analysis. In Proceedings of the International Conference on Artificial Intelligence and Smart Systems, ICAIS, Tamilnadu, India, 25–27 March 2021; pp. 661–668. [Google Scholar] [CrossRef]